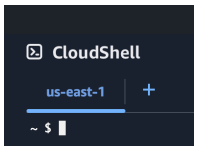
Dquez Carr
MCY 611
Lab 4 AWS ELB & CW

MCY 611 Cloud Computing
Lab#4: AWS Elastic Load Balancing and CloudWatch

0.Open an AWS CloudShell terminal in the us-east-1 region. **Provide a screenshot of your AWS CloudShell terminal in the us-east-1 region.**



Part 1: ELB

1.Create a load balancer via the following command:
aws elb create-load-balancer --load-balancer-name *your_username* --listeners "Protocol=HTTP,LoadBalancerPort=80,InstanceProtocol=HTTP,InstancePort=80" --subnets *your_subnet_id* --security-groups *your_security_group_id*

What is your command? What is the DNS_Name of the load balancer?

```
~ $ aws elb create-load-balancer --load-balancer-name carrd12 --listeners "Protocol=HTTP,LoadBalancerPort=80,InstanceProtocol
=HTTP,InstancePort=80" --subnets subnet-0aa6a451bcda4099f --security-groups sg-00c15e3626170b8b6
{
    "DNSName": "carrd12-561141510.us-east-1.elb.amazonaws.com"
}
~ $
```

2.Describe the state and properties of load balancers via the following command:
aws elb describe-load-balancers --load-balancer-name *your_username*

What is your command and its output? What is the AvailabilityZone?
us-east-1a

```
~ $ aws elb describe-load-balancers --load-balancer-name carrd12
{
    "LoadBalancerDescriptions": [
        {
            "LoadBalancerName": "carrd12",
            "DNSName": "carrd12-561141510.us-east-1.elb.amazonaws.com",
            "CanonicalHostedZoneName": "carrd12-561141510.us-east-1.elb.amazonaws.com",
            "CanonicalHostedZoneNameID": "Z35SXDOTRQ7X7K",
            "ListenerDescriptions": [
                {
                    "Listener": {
                        "Protocol": "HTTP",
                        "LoadBalancerPort": 80,
                        "InstanceProtocol": "HTTP",
                        "InstancePort": 80
                    },
                    "PolicyNames": []
                }
            ],
            "Policies": {
                "AppCookieStickinessPolicies": [],
                "LBCookieStickinessPolicies": [],
                "OtherPolicies": []
            },
            "BackendServerDescriptions": [],
            "AvailabilityZones": [
                "us-east-1a"
```

3.Create a shell script called apache-install under your current directory. The content of apache-install is as follows:

#!/bin/bash
Prevent apt from bringing up any interactive queries
export DEBIAN_FRONTEND=noninteractive

Linux distribution update, and Apache installation
apt update -y
apt full-upgrade -y
apt install apache2 -y
apt autoremove
/etc/init.d/apache2 restart

You manually installed the Apache web server in lab 1. You can use the --user-data option of the run-instances command to automatically run a shell script to install the Apache web server during the instance launch time. The --placement option allows you to specify details about where your instance should be launched, such as the Availability Zone and so on. Create two EC2 instances with Apache web servers via the following command:

aws ec2 run-instances --image-id ami-09e67e426f25ce0d7 --count 2 --instance-type t3.micro --key-name *your_username*-key --security-group-ids *your_security_group_id* --subnet *your_subnet_id* --user-data file://apache-install --placement "AvailabilityZone= *your_AvailabilityZone*"
What is the instance id of the first instance? What is the instance id of the second instance?
i-062567c9048486eb3
i-0f0c8cae58dd1b04c

4.Register instances to the LoadBalancer via the following command:
aws elb register-instances-with-load-balancer --load-balancer-name *your_username* --instances *instance1_id instance2_id*
What is the output?

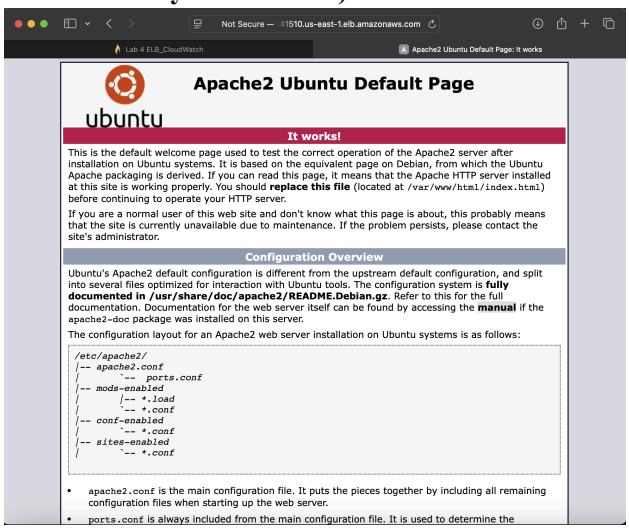
```
~ $ aws elb register-instances-with-load-balancer --load-balancer-name carrd12 --instances i-0f0c8cae58dd1b04c i-062567c90484
86eb3
{
    "Instances": [
        {
            "InstanceId": "i-0f0c8cae58dd1b04c"
        },
        {
            "InstanceId": "i-062567c9048486eb3"
        }
    ]
}
~ $
```

5.Describe the state of the instances via the following command:
aws elb describe-instance-health --load-balancer-name *your_username*
Note: If the state of your EC2 instances is OutOfService, please wait for a couple minutes and try to run the above command again. You should see the state changes to InService.
What is the output?

```
~ $ aws elb describe-instance-health --load-balancer-name carrd12
{
    "InstanceStates": [
        {
            "InstanceId": "i-062567c9048486eb3",
            "State": "InService",
            "ReasonCode": "N/A",
            "Description": "N/A"
        },
        {
            "InstanceId": "i-0f0c8cae58dd1b04c",
            "State": "InService",
            "ReasonCode": "N/A",
            "Description": "N/A"
        }
    ]
}
~ $
```

6.Use a browser to access your load balancer to see if it works. Please type the following in the browser:
http://*dns_name_of_your_load_balancer*/

You can find *dns_name_of_your_load_balancer* at step 1. **What appeared in the browser (provide a screenshot of your browser)?**



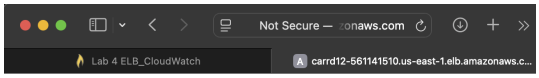
Now let's modify the home page, /var/www/html/index.html, for each web server to differ the contents of the home page so that we can know which web server/instance has served the request. Use ssh to log onto your first instance and switch to root. To access the index.html file, cd to that directory with: cd /var/www/html
Then type into the terminal: vi index.html (enter)
To delete everything that is already in the file: type: 600dd (this will delete 600 lines in the file, which should be all content)
Then press i to insert new content. Copy the html below and paste in the file. Once done, press Esc then :wq

```
<html><body><h1>It works!</h1>
<p>The request was forwarded to instance 1.</p>
<p>The request was served by web server 1.</p>
</body></html>
```

Follow the same step above for the 2nd instance, but this time use the html code below.

```
<html><body><h1>It works!</h1>
<p>The request was forwarded to instance 2.</p>
<p>The request was served by web server 2.</p>
</body></html>
```

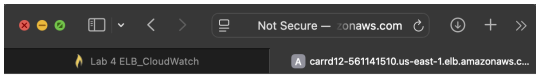
Use your browser to access your load balancer 4 times (you can click the "refresh" button of the browser 4 times to generate 4 requests). **Please tell me how many requests were served by web server 1 and how many requests were served by web server 2? Provide a screenshot of each access.**
There were 2 requests served by web server 1 and 2 requests served by web server 2.



It works!

The request was forwarded to instance 1.

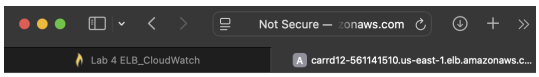
The request was served by web server 1.



It works!

The request was forwarded to instance 2.

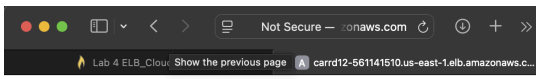
The request was served by web server 2.



It works!

The request was forwarded to instance 1.

The request was served by web server 1.



It works!

The request was forwarded to instance 2.

The request was served by web server 2.

Part 2: CloudWatch

7.Open another CloudShell terminal in the us-east-1 region. Start CloudWatch monitoring on both instances via the following commands:
aws ec2 monitor-instances --instance-ids *instance1_id instance2_id*

What is the output?

```
~ $ aws ec2 monitor-instances --instance-ids i-0f0c8cae58dd1b04c i-062567c9048486eb3
{
    "InstanceMonitorings": [
:...skipping...
{
    "InstanceMonitorings": [
:...skipping...
{
    "InstanceMonitorings": [
        {
            "InstanceId": "i-062567c9048486eb3",
:...skipping...
:...skipping...
{
    "InstanceMonitorings": [
        {
            "InstanceId": "i-062567c9048486eb3",
            "Monitoring": {
:...skipping...
{
    "InstanceMonitorings": [
```

8.Check a list of metrics under the AWS/EC2 namespace via the following:
aws cloudwatch list-metrics --namespace "AWS/EC2"

Did you see the CPUUtilization metric in the output?

```
},
{
    "Namespace": "AWS/EC2",
    "MetricName": "CPUUtilization",
    "Dimensions": [
        {
            "Name": "InstanceId",
            "Value": "i-062567c9048486eb3"
        }
    ]
},
{
    "Namespace": "AWS/EC2",
    "MetricName": "CPUSurplusCreditsCharged",
    "Dimensions": [
        {
            "Name": "InstanceId",
            "Value": "i-0f0c8cae58dd1b04c"
        }
    ]
},
{
```


9.Collect the CPU utilization statistics of the second instance via the following commands:
aws cloudwatch get-metric-statistics --metric-name CPUUtilization --start-time *your_start_time* --end-time *your_end_time* --period 3600 --namespace AWS/EC2 --statistics Maximum --dimensions Name=InstanceId,Value=*instance2_id*
To determine *your_start_time* and *your_end_time*, you can run the "date -u" command to get current time in the UTC format. You can set your_end_time to the current time and set your_start_time to your_end_time minus 30 minutes. For example, start_time is 2021-07-08T23:15:00 and end_time is 2021-07-08T23:45:00.
What is your command and its output?


```
~ $ aws cloudwatch get-metric-statistics --metric-name CPUUtilization --start-time 2025-02-10T01:45:48 --end-time 2025
-02-10T02:15:48 --period 3600 --namespace AWS/EC2 --statistics Maximum --dimensions Name=InstanceId,Value=i-0f0c8cae58
dd1b04c
{
    "Label": "CPUUtilization",
    "Datapoints": [
        {
            "Timestamp": "2025-02-10T01:45:00+00:00",
            "Maximum": 0.1586056062907992,
            "Unit": "Percent"
        }
    ]
}
~ $
```


10.Run an Apache HTTP server benchmarking tool, ab, to test your load balancer. For more information on ab, please look at http://httpd.apache.org/docs/2.0/programs/ab.html
Install the ab tool with the following commands:
sudo yum update -y
sudo yum install -y httpd-tools
ab -V
What is the output of "ab -V"?


```
Complete!
~ $ ab -V
This is ApacheBench, Version 2.3 <$Revision: 1913912 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

~ $
```


ab -n 50 -c 5 *http://your_load_balancer_domain_name/*
What do –n 50 and –c 5 mean? What is the output?
The apache benchmark command ab -n 50 -c 5 sends -n=50 total HTTP request to the specified URL -c=5 simultaneous request at a time.

```
~ $ ab -n 50 -c 5 http://carrd12-561141510.us-east-1.elb.amazonaws.com/
This is ApacheBench, Version 2.3 <$Revision: 1913912 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking carrd12-561141510.us-east-1.elb.amazonaws.com (be patient).....done


Server Software:        Apache/2.4.41
Server Hostname:        carrd12-561141510.us-east-1.elb.amazonaws.com
Server Port:            80

Document Path:          /
Document Length:        145 bytes

Concurrency Level:      5
Time taken for tests:   0.048 seconds
Complete requests:      50
Failed requests:        0
Total transferred:      20750 bytes
HTML transferred:       7250 bytes
Requests per second:    1043.21 [#/sec] (mean)
Time per request:       4.793 [ms] (mean)
Time per request:       0.959 [ms] (mean, across all concurrent requests)
```

11.Collect the latency metric from the ELB via the following:
aws cloudwatch get-metric-statistics --metric-name Latency --start-time *your_start_time* --end-time *your_end_time* --period 3600 --namespace AWS/ELB --statistics Maximum --dimensions Name=LoadBalancerName,Value=*your_username*

What is your command and its output?

```
~ $ aws cloudwatch get-metric-statistics --metric-name Latency --start-time 2025-02-10T01:45:48 --end-time 2025-02-10T
02:15:48 --period 3600 --namespace AWS/ELB --statistics Maximum --dimensions Name=LoadBalancerName,Value=carrd12
{
    "Label": "Latency",
    "Datapoints": [
        {
            "Timestamp": "2025-02-10T01:45:00+00:00",
            "Maximum": 0.0009903907775878906,
            "Unit": "Seconds"
        }
    ]
}
~ $
```

aws cloudwatch get-metric-statistics --metric-name RequestCount --start-time *your_start_time* --end-time *your_end_time* --period 3600 --namespace AWS/ELB --statistics Sum --dimensions Name=LoadBalancerName,Value=*your_username*

How many times have you accessed your load balancer so far, including both browser accesses and ab command accesses? What is your command and its output (the request count in the output should reflect the total number of accesses. If it does not, please check the start time and end time in your command.)?

Sum=8

```
~ $ aws cloudwatch get-metric-statistics --metric-name RequestCount --start-time 2025-02-10T01:45:48 --end-time 2025-0
2-10T02:15:48 --period 3600 --namespace AWS/ELB --statistics Sum --dimensions Name=LoadBalancerName,Value=carrd12
{
    "Label": "RequestCount",
    "Datapoints": [
        {
            "Timestamp": "2025-02-10T01:45:00+00:00",
            "Sum": 8.0,
            "Unit": "Count"
        }
    ]
}
~ $ aws elb deregister-instances-from-load-balancer --load-balancer-name carrd12 --instances i-0f0c8cae58dd1b04c i-062
567c9048486eb3
{
    "Instances": []
}
~ $
```

Part 3: Cleanup

12.Deregister instances from the ELB via the following command:
aws elb deregister-instances-from-load-balancer --load-balancer-name *your_username* --instances *instance1_id instance2_id*

What is your command and its output?

```
~ $ aws cloudwatch get-metric-statistics --metric-name RequestCount --start-time 2025-02-10T01:45:48 --end-time 2025-0
2-10T02:15:48 --period 3600 --namespace AWS/ELB --statistics Sum --dimensions Name=LoadBalancerName,Value=carrd12
{
    "Label": "RequestCount",
    "Datapoints": [
        {
            "Timestamp": "2025-02-10T01:45:00+00:00",
            "Sum": 8.0,
            "Unit": "Count"
        }
    ]
}
~ $ aws elb deregister-instances-from-load-balancer --load-balancer-name carrd12 --instances i-0f0c8cae58dd1b04c i-062
567c9048486eb3
{
    "Instances": []
}
~ $
```

13.Delete the ELB via the following command:
aws elb delete-load-balancer --load-balancer-name *your_username*

14.Terminate your instances. **What command did you use? What is the output?**

```
~ $ aws ec2 terminate-instances --instance-ids i-062567c9048486eb3 i-0f0c8cae58dd1b04c
{
    "TerminatingInstances": [
:...skipping...
{
    "TerminatingInstances": [
        {
            "InstanceId": "i-0f0c8cae58dd1b04c",
:...skipping...
{
    "TerminatingInstances": [
        {
            "InstanceId": "i-0f0c8cae58dd1b04c",
            "CurrentState": {
                "Code": 32,
:...skipping...
{
    "TerminatingInstances": [
```

Submission instructions
After completing this work, submit this word document with your answers to Canvas. In your word document, your answers should be in **bold** print. At the end of your word document, you should write a few sentences that explain the goals of this lab and a paragraph or two that summarizes (high level) the steps one needs to take in order to complete those goals.

This lab's objective is to use the AWS CLI to comprehend and operate with Amazon CloudWatch and Amazon Elastic Load Balancing (ELB). While CloudWatch offers logging and monitoring capabilities for AWS resources, ELB assists in distributing incoming traffic among several instances to guarantee high availability and fault tolerance.
In this lab, I set up Amazon CloudWatch monitoring and created and configured an Elastic Load Balancer (ELB) using the AWS CLI. To make sure traffic was only sent to healthy instances, I set up health checks, specified target groups, and registered EC2 instances with the ELB. After that, I obtained CloudWatch metrics, set up notifications for particular events, and made alarms to keep an eye on important performance metrics like request latency and instance health. In order to confirm that the ELB distributed requests efficiently and that the monitoring tools appropriately recorded system performance, I lastly tested the configuration by creating traffic and examining CloudWatch data.